1주차 예습과제

📍ResNet - Deep Residual Learning for Image Recognition

1. Introduction
    논문이 다루는 분야
    해당 task에서 기존 연구 한계점
    논문의 contributions
2. Related Work
3. 제안 방법론
    Main Idea
    Contribution
4. 실험 및 결과
    Dataset
    Baseline
    결과
5. 결론 (배운점)

1. Introduction

💡 논문에서 다루고 있는 주제가 무엇인지와 해당 주제의 필요성이 무엇인가
논문에서 제안하는 방법이 기존 방법의 문제점에 대응되도록 제안 되었는가

논문이 다루는 분야

- Image classification에서 사용되는 Deep convolutional neural networks
- 네트워크의 깊이가 중요하다

  → ***더 많은 레이어를 쌓는 것만으로 더 신경망을 잘 학습시킬 수 있을까?***

해당 task에서 기존 연구 한계점

1. **vanishing/exploding gradients 기울기 소실/폭주 문제**
    - 정규화된 가중치 초기화, 중간 정규화 레이어를 통해 해결

      → 역전파 기반 SGD로 학습할 때 수십 개의 레이어를 가진 신경망도 안정적으로 수렴
2. **degradation problem 성능 저하 문제**

- 네트워크 깊이가 늘어나면, 정확도가 포화되고, 그 후 급격히 낮아진다
    - overfitting으로 인한 문제 X
- 충분히 깊은 모델에 층을 더 추가 → 더 높은 training error

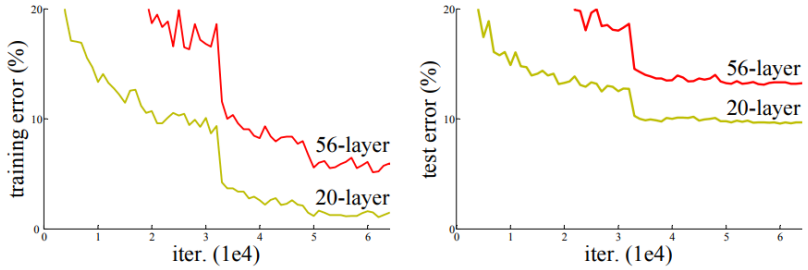

Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

- 성능 저하 문제는 모든 신경망이 동일하게 최적화되기 쉽지 않다는 것을 보여줌

  ex) 얕은 구조가 있고 그 위에 레이어를 더 추가해 만든 더 깊은 모델을 만든 경우

    - 이론적으로는 깊은 모델이 얕은 모델과 동일한 해를 구성적으로 가질 수 있어야 한다(solution by construction, constructed solution)
      ⇒ 즉, 동일한 성능을 낼 수 있어야 하지만 성능 저하가 발생한다.
        - 이때 추가된 층들은 identity mapping(입력을 그대로 다음 층으로 전달하는 것)을 수행, 나머지 층들은 얕은 모델 구조에서 이미 학습된 가중치를 그대로 복사해 사용

⇒ 이와 같이 구성된 해가 존재한다는 것은 깊은 모델의 오차가 얕은 모델의 오차보다 낮아야 한다는 것을 의미하지만, 현재 우리의 최적화 방법들은 이 구성된 해만큼 좋거나 그보다 나은 해를 찾아내지 못하고 있음

논문의 contributions

- **Deep residual learning framework**
    - degradation 문제의 해결책
    - 몇 개의 레이어를 쌓아 올린 층들이 기본 매핑(underlying mapping), 즉 목표 함수를 직접 학습하는 대신, 이 레이어들이 잔차 매핑(residual mapping)을 학습하도록 설계

📌 원래 학습하고 싶은 함수(기본 매핑): **H(x)**
잔차 함수(residual): **F(x)** 라고 할 때,
쌓여 있는 비선형 레이어들이 **F(x) := H(x) - x** 를 학습하도록 한다.
⇒ 원래의 매핑이 **H(x) = F(x) + x** 로 다시 표현된다.

- residual mapping이 최적화하기에 더 쉽다 (ex) 최적 함수가 identity mapping(H(x)=x)인 경우)

- **Shortcut connections**
  - F(x) + x 구조는 shortcut connections(지름길 연결)를 사용한 순전파 신경망으로 구현할 수 있다
  - Shortcut connections
    - 하나 이상의 레이어를 건너뛰는 연결
    - 여기서는 identity mapping을 수행하며, 출력은 쌓여 있는 레이어들의 출력에 더해짐

      ⇒ 레이어를 건너뛰고 그대로 x를 전달, 마지막에 F(x) + x 처럼 두 값을 더함

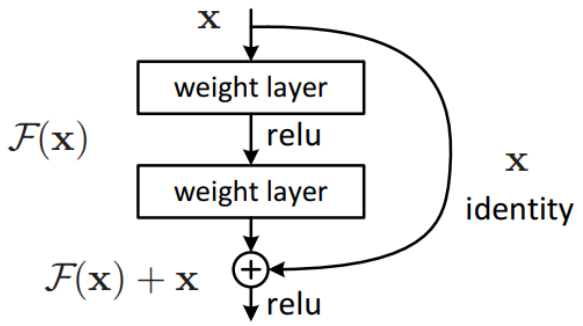

Figure 2. Residual learning: a building block.

  - 단순히 입력 x를 그대로 전달하므로 추가적인 파라미터나 계산 복잡도를 증가시키지 X
  - 기존 SGD + backpropagation 그대로 사용 가능, 일반적인 라이브러리를 사용해 쉽게 구현 가능

- ImageNet에서 실험을 수행
  1. extremely deep residual nets은 최적화가 쉽지만, 단순히 레이어만 쌓은 plain nets은 깊이가 증가할수록 높은 training error를 보임

2. deep residual nets은 깊이를 크게 증가시켜도 쉽게 정확도를 향상시킬 수 있으며, 이전의 네트워크들보다 더 좋은 성능을 보임

2. Related Work

💡 Introduction에서 언급한 기존 연구들에 대해 어떻게 서술하는가
제안 방법의 차별성을 어떻게 표현하고 있는가

- **Residual Representations.**
  - VLAD
    - 사전에 정의된 딕셔너리에 대한 residual vectors를 이용해 인코딩하는 표현 방법
  - Fisher Vector
    - VLAD의 확률적 버전

    ⇒ 두 가지 모두 이미지 검색과 분류에서 강력한 shallow representation 방법

    - vector quantization에서 original vectors를 인코딩하는 것보다 residual vectors를 인코딩하는 것이 더 효율적이라는 연구 결과가 존재
  - Multigirid method
    - 저수준 비전(low-level vision)과 컴퓨터 그래픽스 분야에서 편미분 방정식(Partial Differential Equations, PDEs)을 해결하기 위해 사용되는 방법
    - 시스템을 여러 스케일의 하위 문제들로 재구성, 이때 각 하위 문제는 더 거친 해상도(coarser scale)과 더 세밀한 해상도(finer scale) 사이의 잔차(residual) 해(현재 해와 실제 해 사이의 차이를 보정하는 해)를 담당
  - Hierarchical basis preconditioning
    - Multigrid의 대안
    - 두 스케일 사이의 잔차 벡터(residual vector)를 나타내는 변수들을 사용
    - 해의 잔차적 특성을 고려하지 않는 일반적인 solver들보다 훨씬 더 빠르게 수렴

  ⇒ reformulation이나 preconditioning은 최적화 과정을 더 단순하게 만들 수 있다, 즉 residual의 형태로 문제를 바꾸면 최적화가 쉬워진다.

- **Shortcut Connections.**
  - 다층 퍼셉트론(multi-layer perceptrons, MLPs) 학습의 초기 방법
    - 네트워크의 입력에서 출력으로 직접 연결되는 선형 레이어를 추가
  - 기울기 소실/폭주 문제를 해결하기 위해 중간 레이어를 보조 분류기(auxiliary classifier)에 직접 연결
  - shortcut connection 기반 normalization/centering
  - Inception layer: shortcut branch와 여러 개의 더 깊은 branch로 구성
  - Highway networks
    - gating function을 사용한 shorcut connection
    - data-dependent, 파라미터를 가짐
    - 게이트가 닫히게(값이 0에 가까워지게) 되면, 그 레이어들은 residual 함수가 아닌 일반적인(non-residual) 함수를 표현

    — identity shortcuts과의 차이점
    - identity shortcuts은 파라미터를 가지지 않음
    - 항상 residual function을 학습, 닫히지 않음
    - highway networks는 매우 깊은 구조에서 정확도 향상 X

3. 제안 방법론

💡 Introduction에서 언급된 내용과 동일하게 작성되어 있는가
Introduction에서 언급한 제안 방법이 가지는 장점에 대한 근거가 있는가
제안 방법에 대한 설명이 구현 가능하도록 작성되어 있는가

Main Idea

3.1. Residual Learning

- 기존 방식: 네트워크가 직접 기본 매핑 함수 H(x)를 학습
- Residual learning: 잔차 **F(x) = H(x) - x**를 학습

  → 최종적으로 원래의 함수는 **H(x) = F(x) + x** 가 됨
  - 두 방식 모두 이론적으로 원하는 함수를 점근적으로 근사할 수 있지만, residual 방식의 최적화(학습)가 더 쉬울 수 있다.

- degradation 문제 해결을 위한 reformulation
  - 이론적으로 깊은 모델 = 얕은 모델 + 추가 레이어(identity mapping)이라면 더 깊은 모델의 training error가 얕은 모델보다 낮아야 함
  - 하지만 여러 nonlinear layers로 identity mapping을 근사하는 것이 어렵기 때문에 degradation 문제 발생
  - 이때 H(x) 대신 **residual F(x)=H(x)−x** 학습, identity mapping이 최적해라면 (H(x)=x라면) **F(x)=0** 만 만들면 되므로 weights → 0 으로 쉽게 학습 가능
- 실제 학습 상황에서 이 reformulation은 문제를 더 잘 precondition 가능
  - identity mapping이 최적일 가능성, 즉 H(x)=x 가 완전히 최적일 가능성은 낮음
  - 만약 최적 함수가 zero mapping보다 identity mapping에 더 가깝다
    → solver가 완전히 새로운 함수를 학습하는 것보다 identity mapping을 기준으로 두고 작은 변화(perturbation)만 학습하면 됨

> 📌 **H(x) = x + F(x)**
> (x : 기본 값 (identity), F(x) : 작은 보정(residual)이 됨)

  - 실험 결과 실제로 residual 함수 F(x)가 작은 값 갖는 경우 많음

⇒ identity mappping이 합리적인 preconditioning을 제공, optimization을 더 쉽게 만든다

3.2. Identity Mapping by Shortcuts

- 몇 개의 레이어가 쌓인 구조마다 residual learning을 적용

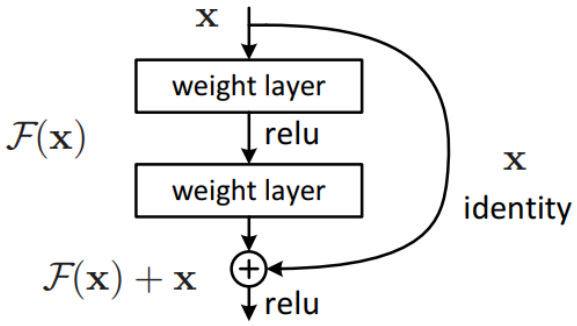

Figure 2. Residual learning: a building block.

Building block을 다음과 같이 정의: Eqn.(1)

$$y = F(x, \{W_i\}) + x$$

- x와 y는 해당 레이어들의 입력과 출력 벡터
- 함수 $F(x, \{W_i\})$는 학습해야 할 residual mapping
- 두 개의 레이어가 있는 경우에는 $F = W_2\sigma(W_1x)$로 표현, σ는 ReLU 의미
- F+x 연산은 shortcut connection과 element-wise addition을 통해 수행
- 이 덧셈 이후에 두 번째 비선형 함수(즉, σ(y), Fig. 2 참고)를 적용

- Residual network의 특징
  - shortcut connections는 추가적인 파라미터나 계산 복잡도를 증가시키지 X
  - plain network와 파라미터 수, 깊이, 넓이, 계산 비용이 동일(element-wise addition만 추가)

📌 Eqn.(1)에서 F와 x의 차원은 같아야 함

- 차원이 다른 경우 (채널 수 증가 등)
  - shortcut connection에서 linear projection W_s 사용해 차원 맞출 수 있음: Eqn.(2)

$$y = F(x, \{W_i\}) + W_s x$$

- shortcut을 $W_s x$로 변환해 차원 맞춤

- Residual function F의 구조
  - 보통 2~3개의 레이어로 구성, 더 많은 레이어도 가능
  - 만약 1개의 레이어만 가진다면 Eqn.(1)은 선형 레이어 $y = W_1 x + x$와 유사해짐 → 큰 장점 X
  - fully-connected layer 기준으로 작성되었지만 convolutional layer에도 적용 가능
  - 함수 $F(x, \{W_i\})$는 여러 개의 convolution layer로 구성될 수 있으며, element-wise addition은 두 feature map 사이에서 채널 별로 수행

3.3. Network Architectures

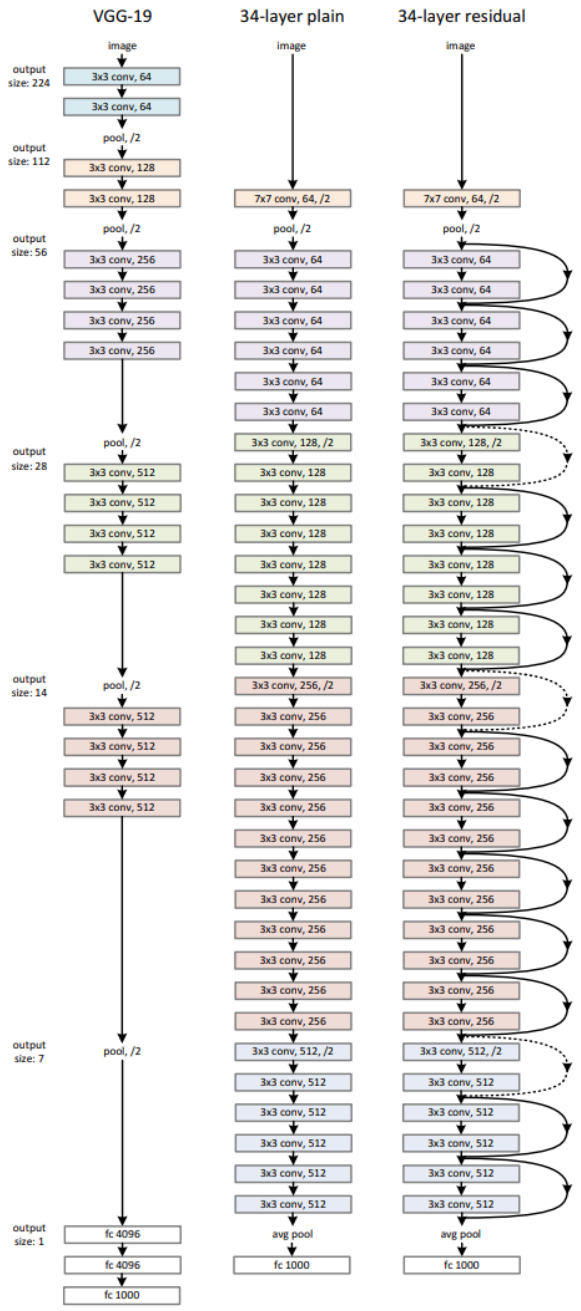

Figure 3. Example network architectures for ImageNet. **Left**: the VGG-19 model [40] (19.6 billion FLOPs) as a reference. **Middle**: a plain network with 34 parameter layers (3.6 billion FLOPs). **Right**: a residual network with 34 parameter layers (3.6 billion FLOPs). The dotted shortcuts increase dimensions. **Table 1** shows more details and other variants.

- **Plain Network.**
  - 3×3 convolution 필터 사용
  1. 출력 feature map의 크기 동일 → 필터 수 동일
  2. feature map의 크기 절반으로 줄어듦 → 레이어당 연산 복잡도를 유지하기 위해 필터 수 두 배
  - stride=2인 convolution layer으로 downsampling
  - 네트워크의 마지막에는 global average pooling layer와 1000개의 클래스 가지는 fully-connected layer(softmax) 위치
  - 여기서는 총 34개의 가중치 레이어(weighted layers)로 구성됨
  - VGG nets보다 필터 수가 적고 계산 복잡도 낮음 → 3.6 billion FLOPs(곱셈-덧셈 연산), VGG-19의 연산량(19.6 billion FLOPs)의 약 18%
- **Residual Network.**
  - Plain Network를 기반으로 shortcut connection을 삽입
  - 입출력 차원이 같은 경우에는 identity shortcut을 그대로 사용
  - 차원이 증가하는 경우 (점선으로 표시)
    1. identity shortcut을 유지, 추가 제로 패딩을 수행해 차원 증가를 맞춤(추가 파라미터 필요 X)
    2. Eqn.(2)의 projection shortcut을 사용해 차원을 맞춤 (1x1 convolution을 통해 수행)

- Feature map 크기가 달라지는 경우에는 stride=2를 사용해 downsampling 수행

3.4. Implementation

- Training
  1. 데이터 전처리
     - shorter side를 256~480으로 Random Resize
     - 224×224 random crop
     - horizontal flip
     - mean subtraction
     - color augmentation
  2. 네트워크 설정
     - Conv → BatchNorm → ReLU
     - weight initialization, scratch training
  3. 학습 설정
     - Optimizer = SGD
     - Batch size = 256
     - Initial LR = 0.1, LR schedule → plateau 시 1/10 감소
     - Iterations = 60×10^4
     - Weight decay = 0.0001
     - Momentum = 0.9
     - Dropout은 사용 안 함

- Testing
  - 10-crop testing
  - fully convolutional form
  - average the scores at multiple scales (resize image with {224, 256, 384, 480, 640})

Contribution

1. **Residual Learning Framework 제안**

- degradation problem 해결을 위해 residual function F(x)=H(x)-x 학습

2. **Shortcut Connection 기반 Residual Block 설계**

- identity mapping을 활용한 residual block
- 추가 파라미터 없이 깊은 네트워크 학습 가능

4. 실험 및 결과

💡 Introduction에서 언급한 제안 방법의 장점을 검증하기 위한 실험이 있는가

Dataset

4.1. ImageNet Classification

- 1000개의 클래스로 구성된 ImageNet 2012 classification dataset에서 평가
  - 1.28 million training images, 50k validation images, 100k test images
  - top-1, top-5 error rates로 평가

4.2. CIFAR-10 and Analysis

- CIFAR-10 데이터셋에서 추가적인 실험 수행
  - **10개의 클래스**로 구성, **50k training images, 10k testing images**

Baseline

4.1. ImageNet Classification

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Table 1. Architectures for ImageNet. Building blocks are shown in brackets (see also Fig. 5), with the numbers of blocks stacked. Down-sampling is performed by conv3_1, conv4_1, and conv5_1 with a stride of 2.

- **Plain Networks.**
  - 18-layer와 34-layer plain nets를 평가
- **Residual Networks.**
  - 기본 구조는 plain nets와 동일, 각 3x3 filters 쌍에 shortcut connection을 추가
  - 첫 번째 비교(Table 2와 Fig. 4 오른쪽)에서는 모든 shortcut에 identity mapping을 사용, 차원이 증가하는 경우에는 zero-padding(option A)을 사용 → plain에 추가적인 파라미터 X

- **Identity vs. Projection Shortcuts.**
  - **(A)** 차원이 증가할 때 zero-padding shortcut 사용, 모든 shortcut은 파라미터 없음
  - **(B)** 차원이 증가할 때는 projection shortcut 사용, 나머지 shortcut은 identity
  - **(C)** 모든 shortcut을 projection으로 사용

- **Deeper Bottleneck Architectures.**

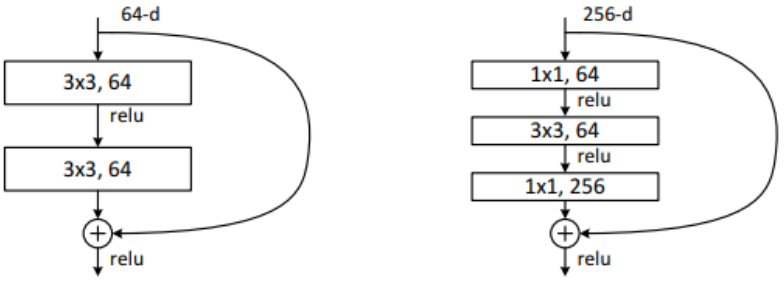

Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a "bottleneck" building block for ResNet-50/101/152.

- residual function F에 대해 2개 대신 3개의 레이어 스택을 사용
  - 1×1 convolution, 3×3 convolution, 1×1 convolution으로 구성
    - 첫 번째 1×1: 차원 축소 (dimension reduction)
    - 3×3: 작은 차원에서 연산 수행 (bottleneck)
    - 마지막 1×1: 차원 복원 (dimension restoration)
- 두 설계가 비슷한 시간 복잡도(time complexity)를 가짐
- 파라미터가 없는 identity shortcut은 bottleneck 구조에서 더 효율적인 모델을 만들도록 해줌

- **50-layer ResNet:**
  - 34-layer 네트워크의 각 2-layer 블록을 3-layer bottleneck 블록으로 교체하여 50-layer ResNet을 구성
  - 차원을 증가시킬 때는 옵션 B 사용
- **101-layer and 152-layer ResNets:**
  - 3-layer 블록을 더 사용해 101-layer와 152-layer ResNet을 구성

4.2. CIFAR-10 and Analysis

- plain/residual 구조는 Fig.3 따름
  - input = 32×32 image, mean subtracted
  - first layer = 3×3 convolutions

- 중간 레이어: {32, 16, 8} 크기의 피처 맵(Feature map)에 대해 각각 2n개씩, 총 6n개의 레이어가 스택으로 쌓임
- 필터 수 = 피처 맵 크기에 따라 각각 {16, 32, 64}개
- Subsampling = convolutions with a stride=2
- 전역 평균 풀링(Global average pooling), 10개 분류를 위한 완전 연결 레이어(Fully-connected layer), 그리고 소프트맥스(Softmax)로 끝남

  → 가중치가 있는 레이어 총 6n+2개

output map size	32×32	16×16	8×8
# layers	$1+2n$	$2n$	$2n$
# filters	16	32	64

- Shortcut Connections을 사용할 때, 3×3 레이어 쌍에 연결 (총 3n개의 shortcut)
  - identity shortcuts 사용 → residual model의 깊이, 넓이, 파라미터 개수 plain과 같음
- 학습 설정
  - weight decay = 0.0001, momentum = 0.9
  - weight initialization, Batch Normalization
  - dropout은 사용하지 않음
  - 두 개의 GPU에서 미니배치 크기 128로 학습
  - Initial LR = 0.1, 32k와 48k iteration에서 10으로 나눔, 64k iteration에서 학습 종료
- 데이터 전처리
  - 4 pixel padding
  - Horizontal flip
  - 32×32 random sampling crop

결과

4.1. ImageNet Classification

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	**25.03**

Table 2. Top-1 error (%, 10-crop testing) on ImageNet validation. Here the ResNets have no extra parameter compared to their plain counterparts. Fig. 4 shows the training procedures.

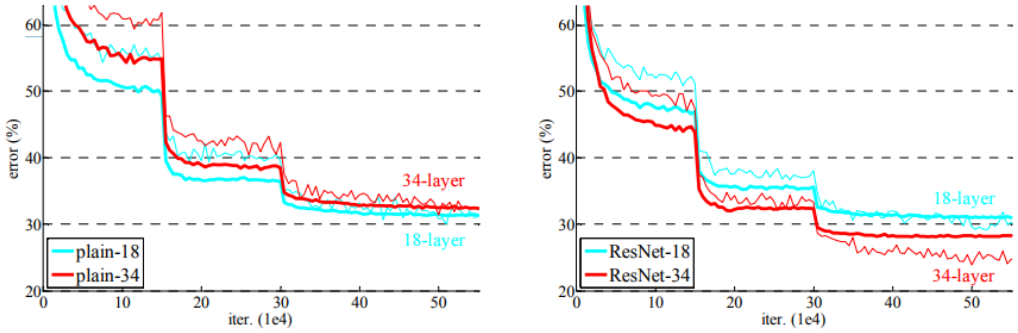

Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

- **Plain Networks.**
  - 34-layer가 18-layer보다 더 높은 validation error를 보임
  - degradation problem 발생 → 34-layer가 전체 학습 과정 동안 18-layer 네트워크보다 더 높은 training error를 보임
  - 깊은 plain 네트워크는 매우 느린 수렴 속도를 가져 training error 감소에 영향을 미친 것으로 추측
- **Residual Networks.**
  - 관찰 결과
    1. residual learning을 사용하면 34-layer가 18-layer보다 더 좋은 성능을 보인다(2.8% 향상)
       - 34-layer ResNet이 훨씬 낮은 training error를 보임, validation 데이터에도 잘 일반화

         → **degradation problem이 해결**됨
    2. 34-layer ResNet은 plain 네트워크와 비교했을 때 top-1 error 3.5% 감소
       - residual learning이 매우 깊은 네트워크에서 효과적이다
    3. 18-layer에서 plain과 ResNet은 정확도가 거의 비슷하지만 18-layer ResNet이 더 빠르게 수렴

- 네트워크가 너무 깊지 않은 경우 ResNet이 초기 학습 단계에서 더 빠른 수렴 보임

- **Identity vs. Projection Shortcuts.**

model	top-1 err.	top-5 err.
VGG-16 [40]	28.07	9.33
GoogLeNet [43]	-	9.15
PReLU-net [12]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	**21.43**	**5.71**

Table 3. Error rates (%, **10-crop** testing) on ImageNet validation. VGG-16 is based on our test. ResNet-50/101/152 are of option B that only uses projections for increasing dimensions.

  - 세 방법 모두 plain network보다 좋은 성능 보임
  - A/B/C 간의 차이가 매우 작음 → degradation 문제를 해결하는 데 projection shortcut이 필수적이지 X
  - identity shortcut은 bottleneck 구조의 복잡도를 증가시키지 않는다는 점에서 중요

- **50-layer ResNet:**
  - 모델 연산량 = 3.8 billion FLOPs
- **101-layer and 152-layer ResNets:**
  - 네트워크 깊이가 크게 증가했음에도 152-layer ResNet (11.3 billion FLOPs)이 여전히 VGG-16 / VGG-19 네트워크 (15.3 / 19.6 billion FLOPs)보다 더 낮은 연산 복잡도를 가짐
  - 50 / 101 / 152-layer ResNet은 34-layer 네트워크보다 더 높은 정확도를 보임

- degradation 문제 X → 깊이의 증가로 정확도 향상

model	top-1 err.	top-5 err.
VGG-16 [40]	28.07	9.33
GoogLeNet [43]	-	9.15
PReLU-net [12]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	**21.43**	**5.71**

Table 3. Error rates (%, **10-crop** testing) on ImageNet validation. VGG-16 is based on our test. ResNet-50/101/152 are of option B that only uses projections for increasing dimensions.

method	top-1 err.	top-5 err.
VGG [40] (ILSVRC'14)	-	8.43†
GoogLeNet [43] (ILSVRC'14)	-	7.89
VGG [40] (v5)	24.4	7.1
PReLU-net [12]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	**19.38**	**4.49**

Table 4. Error rates (%) of **single-model** results on the ImageNet validation set (except † reported on the test set).

- **Comparisons with State-of-the-art Methods.**
  - 기본 34-layer ResNet은 매우 경쟁력 있는 정확도 달성
  - **152-layer ResNet**은 단일 모델 기준 **top-5 validation error 4.49%**로, 이전의 **모든 앙상블 결과보다 더 뛰어난 성능** 보임
  - 앙상블 모델은 테스트 세트에서 3.57%의 top-5 error를 기록
  - ILSVRC 2015 대회에서 1위

4.2. CIFAR-10 and Analysis

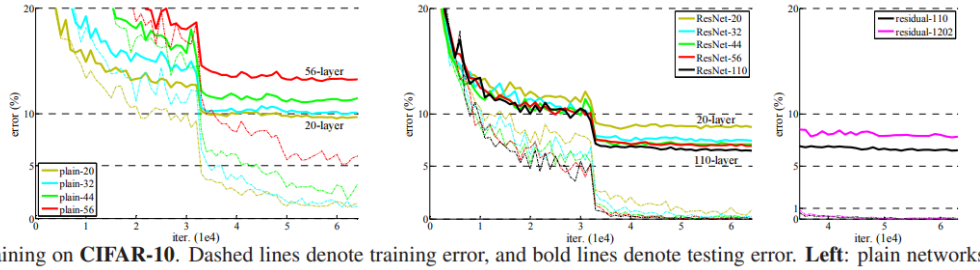

Figure 6. Training on **CIFAR-10**. Dashed lines denote training error, and bold lines denote testing error. **Left**: plain networks. The error of plain-110 is higher than 60% and not displayed. **Middle**: ResNets. **Right**: ResNets with 110 and 1202 layers.

- n = {3, 5, 7, 9} 실험 (20, 32, 44, 56-later networks)
  - Plain net에서는 Degradation Problem 발생, ResNet은 층이 깊어질수록 성능이 비례해서 좋아짐

- 110-layer의 경우(n=18) 초기에 매우 작은 학습률로 시작하여 안정화시킨 뒤 본래 학습률로 올리는 Warm-up 전략 사용해 수렴시킴

  → 기존의 다른 깊은 모델들(FitNet, Highway 등)보다 파라미터 수가 적으면서도 최고 수준(SOTA)의 성능을 달성

- **Analysis of Layer Responses.**

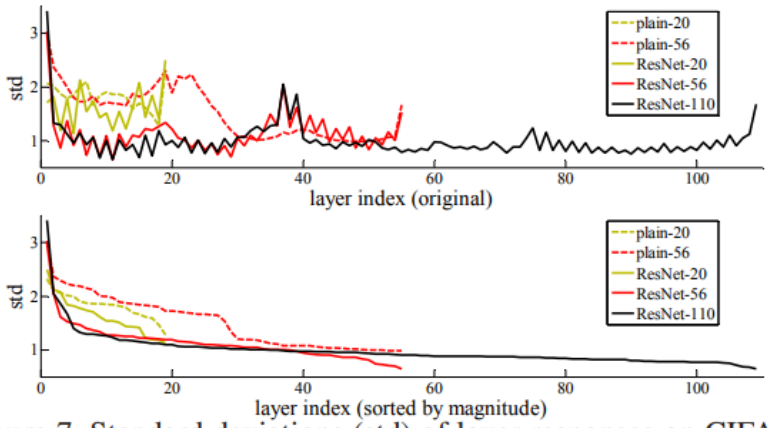

Figure 7. Standard deviations (std) of layer responses on CIFAR-10. The responses are the outputs of each 3×3 layer, after BN and before nonlinearity. **Top**: the layers are shown in their original order. **Bottom**: the responses are ranked in descending order.

  - 레이어 응답의 표준 편차(std) → Residual Function의 응답 강도 보여줌
    - ResNet의 레이어 응답이 0에 가까움 → Shortcut을 통해 입력을 그대로 전달하고 레이어는 잔차만을 학습
    - 모델이 깊어질수록 개별 레이어가 원래의 신호를 변화시키는 정도가 더 작아지는 경향

- **Exploring Over 1000 layers.**
  - **n=200**으로 설정해 **1202-layer network** 구축
    - **최적화 성공적, 0.1% 미만의 Training error**
    - 110-layer와 비슷한 training error 보였지만 테스트 결과 110-layer보다 좋지 않음

      → 모델 용량에 비해 데이터셋이 작아 Overfitting 발생했다 추측

      → 강력한 정규화와 결합한다면 결과 개선 가능할 것

4.3. Object Detection on PASCAL and MS COCO

training data	07+12	07++12
test data	VOC 07 test	VOC 12 test
VGG-16	73.2	70.4
ResNet-101	**76.4**	**73.8**

Table 7. Object detection mAP (%) on the PASCAL VOC 2007/2012 test sets using **baseline** Faster R-CNN. See also appendix for better results.

metric	mAP@.5	mAP@[.5, .95]
VGG-16	41.5	21.2
ResNet-101	**48.4**	**27.2**

Table 8. Object detection mAP (%) on the COCO validation set using **baseline** Faster R-CNN. See also appendix for better results.

- 다른 recognition task에서도 뛰어난 일반화 성능 보임
- VGG-16을 ResNet-101로 교체했을 때 성능 향상
- MS COCO 데이터셋에서 COCO의 표준 평가지표(mAP@[.5, .95]) 기준 6.0% 상승, 대적으로 28% 개선
- ILSVRC & COCO 2015 대회의 여러 부문에서 1위

5. 결론 (배운점)

💡 연구의 의의 및 한계점, 본인이 생각하는 좋았던/아쉬웠던 점 (배운점)

- 깊은 신경망에서 발생하는 degradation problem을 해결하기 위해 residual learning framework를 제안
  - 잔차 함수 F(x) = H(x) - x를 학습하도록 설계
  - identity shortcut connection 도입

  → 추가적인 파라미터 없이도 매우 깊은 신경망 안정적으로 학습 가능
- 152-layer 같이 매우 깊은 네트워크로 뛰어난 성능 보임
- 다양한 딥러닝 모델 구조에 residual connection이 핵심적으로 사용되는 데 큰 영향

- 네트워크 깊이를 증가시켜도 overfitting 문제로 인해 테스트 성능 낮아질 수 있음
  → 데이터 규모, 정규화 기법도 함께 고려되어야 함

🧐 ResNet의 구조와 그 구조를 설계한 이유, ResNet을 통해 어떤 문제를 해결하였는지를 구체적으로 배울 수 있었다.
네트워크 깊이가 깊어진다고 무조건 성능이 향상되는 게 아닌 것처럼, 딥러닝 모델 설계에 있어서 학습이 잘 이루어지도록 네트워크 구조를 설계하는 것이 중요하다는 점을 알 수 있었다. 특히 입력을 그대로 전달하는 shortcut connection 이라는 단순한 아이디어가 매우 깊은 네트워크의 학습을 가능하게 만들었다는 점이 인상적이었다.